

AAROP

Autonomous Agentic Reasoning & Orchestration Platform — a production-grade, multi-agent system built on agentic-loop engineering principles.

Devayan Mandal · AI / ML Engineer · github.com/devtechedge
Reference implementation · Offline-runnable · 92% tested core

Python 3.10+

Multi-Agent Orchestration

Agentic Loop State Machine

9 Tests Passing

92% Core Coverage

Zero API Keys to Run

The Problem

Most "AI agent" demos hide their control flow inside a single sprawling prompt. They look impressive in a recorded clip but collapse the moment a tool fails, a model hallucinates, or cost runs away — because there is no explicit control loop, no verification, and no recovery path.

AAROP demonstrates how an agent system is actually engineered at a senior / chief-architect level: the agentic loop is a first-class, inspectable state machine with bounded autonomy, self-verification, and full trace replay.

The Solution at a Glance

6

LOOP PHASES
(EXPLICIT FSM)

5

PLUGGABLE
SUBSYSTEMS

92%

CORE TEST
COVERAGE

0

API KEYS TO RUN
DEMO

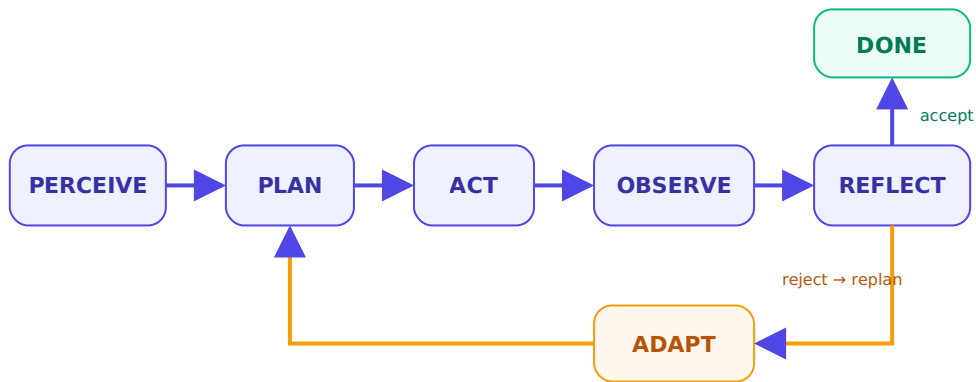
AAROP ingests an open-ended objective and drives it through a closed feedback loop —

Perceive **Plan** **Act** **Observe** **Reflect** **Adapt** — orchestrating specialized tool-using agents,

verifying its own output before committing, and emitting a structured, replayable trace for

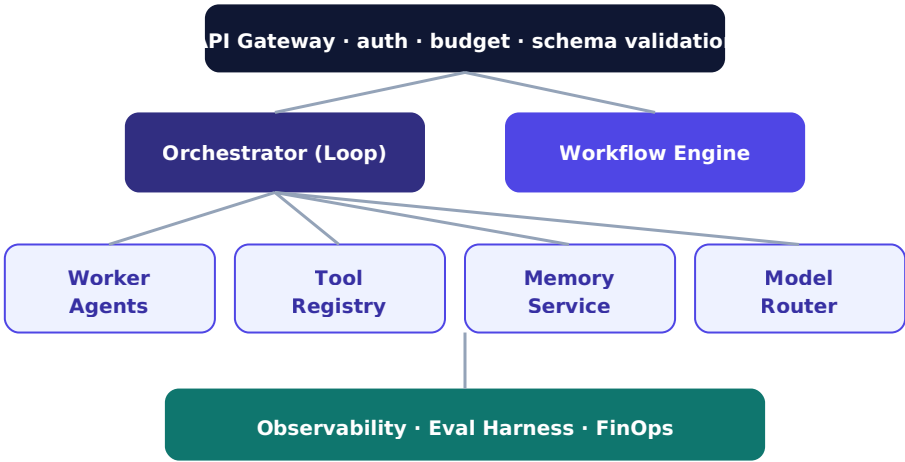
every run. It runs **100% offline** via a deterministic mock model provider, while every backend (model, tools, memory) is pluggable for real deployment.

The Agentic Loop — the heart of the system



Every phase transition emits a structured trace event (`phase_transition` , `tool_call` , `loop_complete`), so any run is reconstructable and replayable deterministically. If a budget is exhausted, the loop escalates to a human instead of looping forever.

System Architecture



Module	Responsibility	Production counterpart
core/loop.py	Agentic loop FSM, Budget guardrails, checkpointing	Temporal durable workflow
agents/agents.py	Planner · Actor · Verifier/Critic	LLM-backed agents (vLLM / OpenAI)
tools/registry.py	Schema-validated tools, scopes, retries, circuit breaker, audit log	Sandboxed (seccomp / gVisor)
memory/store.py	Working / episodic / semantic memory + RAG recall	pgvector / Qdrant + reranker
routing/model_router.py	Cost-aware model selection	Cloud + self-hosted routing
observability/tracing.py	Structured spans/events + replay buffer	OpenTelemetry + Langfuse

Engineering Rigor Demonstrated

Bounded Autonomy

Every run respects step / cost / time budgets and escalates to a human instead of looping forever.

Self-Verification

A critic agent scores each result against acceptance criteria before any commit to memory.

Resilience

Tool retries, circuit breaker, and anomaly detection in the observe phase.

Security Posture

Permission-scoped tools, input validation, sandboxed evaluator, full audit log.

Observability

Full structured trace per run; deterministic replay of any execution.

Testability

All dependencies injected/mockable; 92% coverage on core orchestration.

Verified Results

Running `python examples/run_demo.py --objective "calculate 21*2 + 8"` drives the objective through the full loop and returns the correct result with high confidence:

Phase trace: plan → act → observe → reflect → done

result: "50" · confidence: 0.9 · trace events: 18

Test suite: **9 passed** · coverage **91.6%** (gate: ≥85%) · CI across Python 3.10–3.12.

Key Architecture Decisions (ADRs)

- **ADR-001** — The agentic loop is an explicit state machine, not a hidden prompt chain → enables observability, replay, and crash recovery.
- **ADR-002** — Bounded autonomy via Budget guardrails → predictable cost/latency and a safe failure mode (human escalation).
- **ADR-003** — A Verifier/Critic gate before commit → measurable error reduction over single-pass output.
- **ADR-004** — Pluggable, schema-validated tools with a circuit breaker → resilient, auditable, hot-swappable.
- **ADR-005** — Offline-first, dependency-free core → instantly runnable by reviewers; hermetic CI.

Tech Stack & Production Path

Reference core: **Python** (async-ready, zero runtime deps). Production reference stack documented in the repo: **Temporal** (durable workflows), **FastAPI** + Celery/Arq workers, **gRPC** agent-to-agent protocol, **pgvector/Qdrant** + cross-encoder reranker for RAG, **vLLM/Ollama** for self-hosted models, **OpenTelemetry** + **Langfuse** for tracing, and **Docker** + **Kubernetes** + **Helm** + **Terraform** with KEDA autoscaling.

Devayan Mandal · AI / ML Engineer · github.com/devtechedge/aarop · Full source, architecture doc, ADRs, and runnable demo available in the repository. © 2026.